

智能合约账户 - 账户抽象、模块化、Safe

EVM账户目前有如下几类：

EOA —由个人用户通过私钥控制

问题：

- 私钥难保护：用户失去私钥（遗失、黑客攻击、密码学上的被破解）意味着地失去所有资产。
- 签名算法少：原生协议在验证交易上只能使用 ECDSA 签名和验签算法。
- 签名权限高：无原生多签（多签只能通过智能合约实现协作），单签即可执行任意操作。
- 多币种支付不友好：交易手续费只能通过 ETH 支付。
- 并不支持批量交易。
- 交易隐私泄露：一对一交易容易分析账户持有者的隐私信息。

托管钱包

- 全托管方案 —适用于交易平台、大型机构金服、强合规等场景，比如交易所完全控制用户账户和资金
- 半托管方案 —适用于相对的高端个人用户群体或机构。他们明白服务方可以审查自己的交易，并且有能力提前备份（比如导出私钥），在服务方拒绝服务时不能影响自己的资产安全。这样日常使用时可以享受安全和便利，极端情况下可以保护全部资产。
- 非托管方案 —用户对资金负责，钱包无法自行操作用户的账户，钱包无法阻止用户操作自己的账户

SCW —智能合约钱包（如Safe）

- 以 Safe 多签钱包（原名 Gnosis Safe）为例，可以设定钱包的多签规则，如三人共管、两人签名可执行交易（2-of-3）。
- Argent 钱包的做法也类似，创新点是引入所谓的监护人（Guardian）机制，对用户更友好，用户可以设定其他EOA 账户或邮件/手机作为监护人，监护人可以许可交易、锁定钱包、协助恢复钱包。
- UniPass 是一个让用户无需记住助记词的 Web3 钱包，支持使用 Email 社交恢复方案和无 Gas 体验（UniPass 提供一个默认的中继节点，接受用户使用原生 Token 和主流稳定币形式的 Gas 支付），以及使用零知识证明对链上的用户信息进行脱敏处理。

问题：

- 通过智能合约实现自定义账户逻辑，是账户抽象的早期形式
- 每个用户操作都必须由 EOA 发起，就需要额外消耗 21000 gas 费
- 用户需要多管理个账户，如原生safe钱包需要管理智能合约钱包和一个上链交易用的EOA，存入 ETH 来支付 gas 费，或者依靠中心化中继系统支付

MPC—多方计算MPC

通过分散控制权来达成风控或者灾备能力。目前 Web3 的语境下大都指的是 tss。

TSS: **门限签名**是一种全局多方签名协议，包含密钥生成、签名，以及在不改变公钥的情况下更换私钥碎片的重新共享等算法。

一个 mn 的 tss 指的是一个预估对应了 n 个私钥碎片，其中 m 个碎片的联合签名可以被用于上链验证

元交易标准—openGSN

用户对结构化数据进行签名，数据与签名被发送给中继器（Relayer），链下中继器（某 EOA 账户）向链上发起交易，送到链上可信的传递者合约（Forwarder），而业务合约是定制的，会将这种方式传递的交易中的原始发起者视为是用户，而非 msg.sender

问题：

- 它引入了一个我们必须信任的链下角色中继器（Relayer）。如果他宕机了，或者有作恶的动机（或者私钥被盗），则可能存在风险
- 我们最终交互的合约（即 NFT 合约）必须定制。这导致而无法与链上已经存在的、未定制的合约进行交互，这大幅减少能够适用的范围

账户抽象

ERC-4337账户抽象定义了多个标准

- 通过引入名为 UserOperation 用户操作的标准化了更高层伪交易对象
- 通过加入Bundler进行打包上链来标准化实现交易签名和交易来源的解耦
- 通过定义任意验证逻辑和执行逻辑标准化了智能合约钱包
- 标准化的 Gas 费（燃料费）的代付方式，即代付者（Paymaster）机

这个标准让我们可在不改变共识层的情况下构建账户抽象，这是通过将交易的有效性条件（签名验证、Gas 支付、重放保护）从核心协议层（共识层）转移至 EVM 智能合约层（执行层）来实现的，从而可将所有以太坊账户转变为完全可编程的智能合约。这将为账户所有者、钱包和 dApp 在签署和接受交易方式和交易费来源上提供更大的灵活性。

实现库：

官方：<https://github.com/eth-infinitism>

stackup团队：<https://github.com/stackup-wallet>

EIP 4337 将智能合约钱包及其相关基础设施标准化成多个组件：

UserOperation

用户操作对象，用户发起一个钱包操作，使用任意形式的私钥对用户操作进行签名（指令不变，但内容字段变化，可选用非 ECDSA 签名）产生一个带有签名的用户操作UserOperation

- **sender**：请求的智能合约钱包账户地址。地址可能存在，也可能不存在于链上。如果不存在，将执行 `initCode`。
- **nonce**：唯一值，用来验证发送方是不是在重放操作。通常每次发送请求时只需递增一个。
- **initCode**：如果设置，将使用此数据创建帐户合约
- **callData**：具体调用的参数，to、value、data、operation
- **callGasLimit**：传递给 `callData` 方法调用的 gas 限制。
- **verificationGasLimit**：用于 `validateUserOp` 和 `validatePaymasterUserOp` 的 gas。

- **preVerificationGas**: 不被 handleOps 方法计算的 gas, 但添加到支付的 gas 中。用于覆盖批处理开销。
- **maxFeePerGas**: 与 EIP-1559 gas 参数相同。
- **maxPriorityFeePerGas**: 与 EIP-1559 gas 参数相同。
- **paymasterAndData**: 如果设置, 此字段保存支付主地址和支付主特定数据。支付主将为交易支付费用, 而不是发送方。
- **signature**: 在整个请求中验证发送方的签名, 验证入口点地址和链 ID。

UserOperation 内存池

UserOperation 内存池是等待网络处理 UserOperations 的专用内存池, 是任何 UserOperation 上链成为一笔真正交易之前的最后一站。在这里, 待处理的 UserOperation 将被「捆绑器」监听、获取并打包, 「捆绑器」会替代 EOA 支付 Gas 费, 并将其发送到共识层执行交易。

Bundler

- Bundler 打包器就是一个 EOA。有了 Bundler 之后, 将用户操作内存池中的用户操作进行打包, 后再单独签名一笔交易来包裹用户的指令, 因此用户无需创建和记住 EOA 私钥也可触发智能合约钱包交易。
- Bundler 的作用: 验证 UserOperation、将一组 UserOperation 对象打包成单个“捆绑事务”。将已验证的 UserOperation 打包后签名广播到公共或私人内存池进行上链。
- Bundler 服务支持私有内存池和公共内存池, 并为私有内存池提供定制选项
- Bundler 有过滤垃圾信息的重要功能, 并且出于自身的经济原因, Bundler 希望尽可能地进行监控, 以确保内存池的安全。
- 对于公共Bundler, 需要 Bundler 根据链下对 Paymaster 过往代付情况的记录选定最优的 Paymaster 而淘汰质量较低的 Paymaster, 这一过程形成了 Paymaster 间的竞争, 有助于提升网络运行效率。
- 在接受一个 `UserOperation` 之前, 捆绑器应该使用 RPC 方式来本地调用 `simulateValidation` 函数, 来验证签名是正确的并且操作已经付费。一个节点/捆绑器应该跳过 (或者不添加到内存池) 验证失败的 `UserOperation`。

Endpoint

- 入口点是一个全局合约, 所有 Bundler 都需要调用它来执行 UserOperation。入口点充当 Bundler 和智能合约钱包之间的中介。
- SmartContractWallet和Payment需要在入口点合约中质押eth, 用于支付给Bundler 手续费, Bundler 在调用 Entry Point 合约时会触发合约里的 handleOps 函数, 这个函数首先会检查钱包是否有足够的 gas 补偿 Bundler
- 入口点合约首先验证钱包是否存在, 用 UserOperation 作为输入参数, 在链上调用智能合约钱包来验证 UserOperation, 检查是否由指定的智能合约钱包地址签名, 并且钱包有足够的 gas 费来补偿 Bundler。如果验证成功, 则根据函数签名执行输入参数调用智能合约钱包进行执行具体操作。无论执行是否成功, 入口点合约都会向 Bundler 支付 gas 费
- 如果 UserOperation 中的内容无法通过 validateOp, 则将自动终止执行, 同时整个过程中产生的 gas 消耗全部由 Bundler 自行承担, 而一旦执行到了 executeOp 函数, 无论因为何种原因导致合约执行终止, 所产生的 gas 消耗均由 Wallet Contract 承担。这一机制既有效的保证 Bundler 可以通过自己的打包行为获取正当收益, 又可以防止恶意 Bundler 发起的 Dos 攻击

SmartContractWallet

- 合约里面有一个 multiCall 函数可以批量处理来自 UserOperation 的 calldata 里面的数据从而大大降低 gas 消耗
- 智能合约钱包具体符合标准定义的 UserOperation 的验证和执行两个方法。会授权给 Endpoint 合约进行调用，验证和执行的逻辑由用户自己定义
validateOp 函数中定义了验证步骤：
- Bundler 链下模拟验证，第一次调用 validateOp，验证 UserOperation 中的签名，并确保代付者有足够的 gas 余额；
- 上链时由入口点合约进行第二次调用 validateOp，是在执行 UserOperation 之前执行链上验证。
- 通过将其分离，Bundler 可以在链下验证 UserOperation，从而过滤掉恶意交易，而无需支付 gas 费，并且针对验证和执行可以设置不同的 gaslimit，根据精细的进行控制

Payment

- Paymaster 定义了智能合约钱包的 gas 抽象逻辑，包括使用 ERC20 同质化代币支付以太坊 gas 费以及无需 gas 费的交易。
- 当 UserOperation 中的 paymasterAndData 参数不为空的时候，将由指定的 Paymaster 合约作为 Bundler 代付 gas。这个过程中，Bundler 会触发 Paymaster 中的 validatePaymasterOp 函数，这个函数的作用是在检查了 Paymaster 中是否有足够余额支持支付 gas 后，按照用户自定义的需求为指定的 UserOperation 支付 gas。
- Paymaster 是由 dApp 部署的智能合约，Paymaster 会在入口点合约中质押 eth，用于交易执行后由入口点合约转账给 bundler，Paymaster 合约提供了 validatePaymasterOp 进行代付校验
- Paymaster 在 ERC4337 中不是必须的存在

Wallet工厂

- Wallet Factory 是创建智能合约钱包的公共合约。当发送的 UserOperation 目标智能合约钱包并不存在，且当 initCode 中填入了钱包工厂地址和新智能合约钱包的参数后，Bundler 将触发相应的 Wallet Factory 创建具有指定参数的智能合约。
- 用户可提交填充了 initCode 的 UserOperation，请求 Bundler 创建 CA 钱包。
- 用户可选择带特定定制参数的 Wallet Factory 来定制他们的 CA 钱包。
- 热门的 Wallet Factory 代码经全面审核，因此，用 Wallet Factory 创建钱包更安全。
- Wallet Factory 需要在入口点上质押 ETH，并持续为 UserOperations 提供良好服务，才能从 Bundler 获取更多流量。

Signature聚合器

- 聚合器是一类特殊的参与者，它利用辅助智能合约处理多个 UserOperation 签名，并将其转换（聚合）为一个签名并执行验证。捆绑的 UserOperations 由聚合器一步验证，而不是单独验证每个签名，从而可实现大规模提高交易处理的效率和成本。最流行的压缩方法是使用 BLS 签名聚合。
- 通过将多个签名合并为一个签名，聚合器可以一步验证多个 UserOperations，从而帮助节省 Gas。
- 当此「用户操作 UO」在链上进行验证时，钱包合约和入口点合约会调用「聚合签名验证 Aggregator」合约，对聚合签名进行验证。
- 由于 ERC4337 中支持非 ECDSA 的签名算法。所以我们首先要将使用不同签名算法的 UserOperations 分类，然后由 Bundler 将同类交易通过 Signature Aggregator 生成一个聚合签名，一次验签便可以将里面包含的交易全部进行验证，这一过程大大降低了 gas 费的消耗。

BLS 算法可以在多个公钥下为多条消息生成的多个签名并聚合成一个签名，大幅降低聚合器解析的计算量

BLS 签名算法和 ECDSA 签名算法都是常见的数字签名算法，但二者之间存在一些差别。

签名效率：BLS 签名算法相对于 ECDSA 签名算法具有更高的签名效率。这是因为 BLS 签名算法只需要对消息进行一次哈希运算，而 ECDSA 签名算法需要对消息进行两次哈希运算。

验证效率：BLS 签名算法相对于 ECDSA 签名算法具有更高的验证效率。这是因为 BLS 签名算法只需要对一个公钥和一个签名进行一次运算，而 ECDSA 签名算法需要对两个大整数进行多次运算。

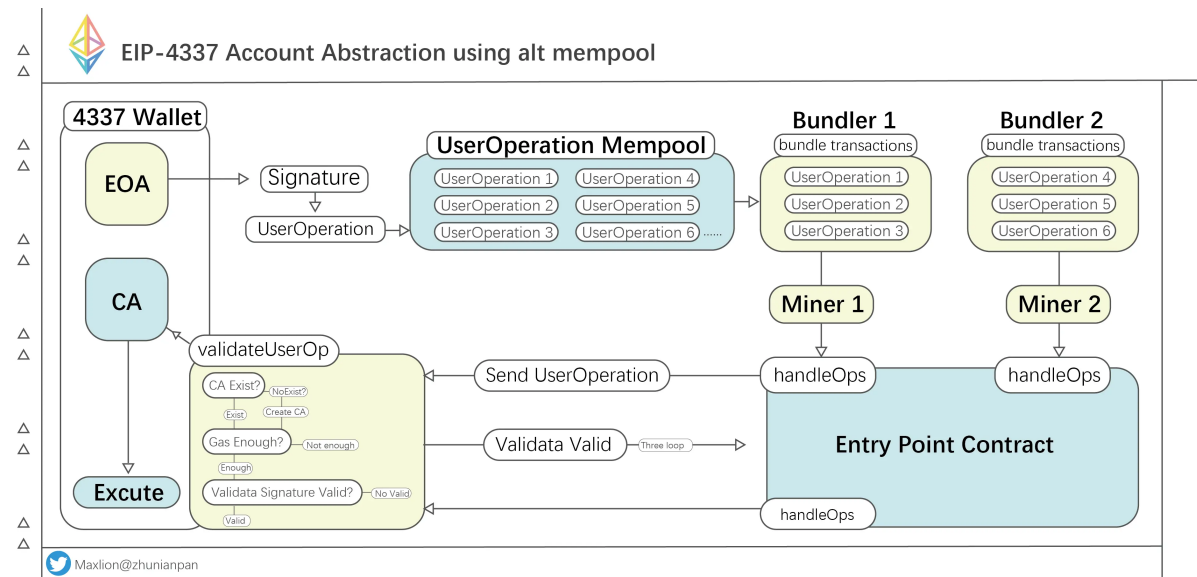
安全性：BLS 签名算法和 ECDSA 签名算法都是基于数学难题实现的，因此它们的安全性都是由数学难题的困难程度决定的。目前来看，BLS 签名算法比 ECDSA 签名算法更加安全，

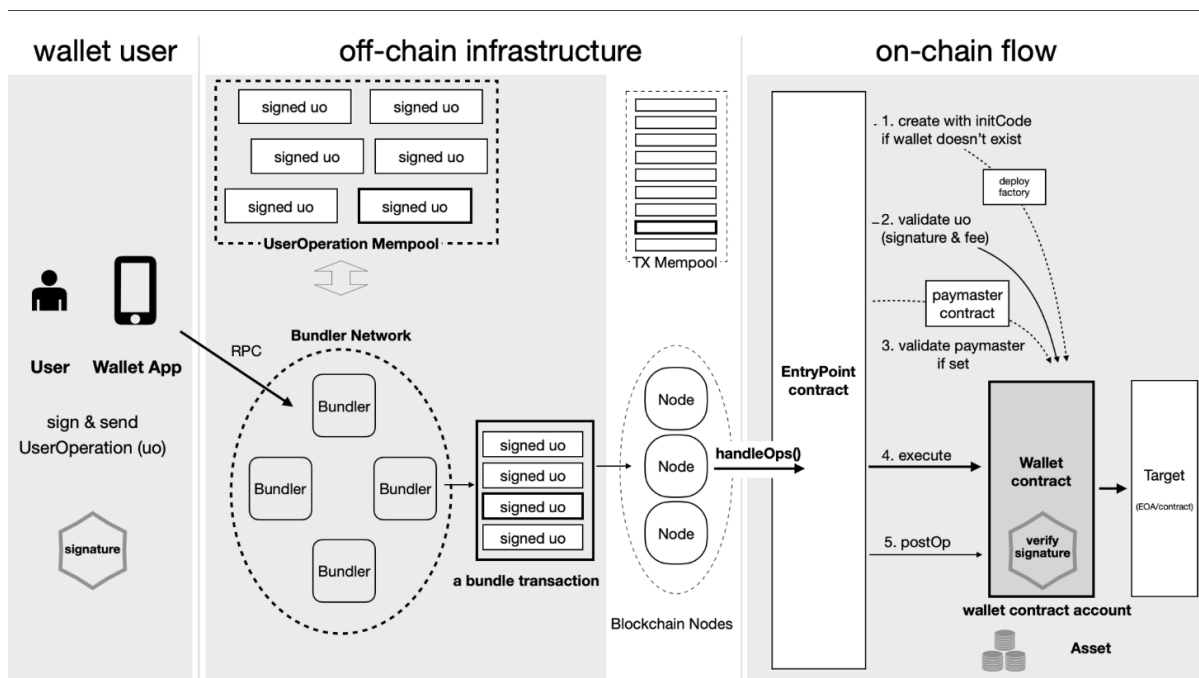
可扩展性：BLS 签名算法相对于 ECDSA 签名算法具有更好的可扩展性。这是因为 BLS 签名算法允许多个签名合并成一个签名，而 ECDSA 签名算法不支持这种功能。

交易步骤：

4337 在未修改以太坊共识层的情况下，引入新操作逻辑 useroperation 并加入 Bundler 负责打包。在原先流程后加入 EntryPoint 和 SmartContractWallet 合约将验证和执行过程拆分来完成自定义任务执行，最后通过 Paymaster 将 gas 费的支付逻辑抽离。

用户发起 userop — userop 被发送到「用户 userop 内存池」— Bundler 监听内存池负责将选中的 UserOps 打包成一笔交易提交给 EntryPoint —（CA 创建）— EntryPoint 验证用户操作 —（validatePaymasterOp）— 智能合约钱包将用户操作执行并纳入区块。





账户抽象优点

- **Gas 抽象:**

Gas 抽象包含无 gas 费交易，并使用任意 ERC20 代币支付 gas 费。该逻辑可在 Paymaster 合同中或通过中继器执行。对 AA 而言，许多智能合约钱包本身可以实现 EIP 4337 兼容的 Paymaster 合约，并在入口点合约上质押代币，帮助用户支付 gas 费。

- **自定义签名密码学方案和多签**

- **社交恢复:**

在私钥丢失或泄露的情况下，用户可以将新密钥授权为合法的钱包所有者。社交登录和社交恢复的逻辑一般在钱包的主合约中定义。可采用多种方式，如电子邮件、多重签名、MPC 或 SWIE（用以以太坊登录）等。

- **交易批处理:**

交易批处理是智能合约钱包独有的功能，允许钱包用户在单个链上交易中执行多个交易。

- **跨链桥接和连接桥整合:**

目前，许多钱包都与第三方供应商合作，将法币充提通道和跨链桥接集成到钱包中。这些充提通道和跨链桥接可以进一步与 gas 抽象中的支付合约（Paymaster）进行整合。

- **模块化设计:**

AA 最大的优势之一可能是其模块化服务，Bundler、Paymaster 和其他部分可以灵活地组合在一起。

- **无共识修改:**

EIP-4337 可在无需更新以太坊核心协议的情况下实现较大程度的账户抽象

- **高度可编程:**

基本实现账户抽象的目的，可以对钱包进行丰富的自定义。a) 签名自定义，可以支持 ECDSA 之外的签名算法，聚合签名；b) 验证自定义，如黑白名单；c) 执行自定义，可以支持如 gas 代付、多签、交易管理、社会恢复、隐私保护等在内的多种功能。任何新加的功能理论上在这个 4337 流程中加一个模块就可以实现。

- **去中心：**

没有一个实体可以影响整个流程的运转，一切都是通过点对点内存池完成的。

- **可升级：**

钱包可更改自己的 public keys，如果使用了代理调用，则可升级整体代码。

账户抽象缺陷

- **手续费（也许）相对较高**

使用 ERC-4337 进行简单转账的成本要比使用传统钱包（通常称为 EOA）高得多，因为前者需要调用合约。

然而，在 Rollup 网络上，使用 ERC-4337 进行简单转账可能比 EOA 更便宜，因为它可以将签名聚合在一起，以减少主网上的数据量。

- **模块堆积，目前尚未最终确定的标准**

随着功能不断增加，可能存在安全漏洞。扩展的交易可扩展性导致攻击向量增加、迁移到新标准时可能出现未知错误或安全风险、需要强大而安全的全局入口点合约来确保所有交易经适当签署及验证，诸如此类的挑战。

- **单点性**

将 EntryPoint 的地址写死在了每一个生成的智能合约账户主体中，如果要对 EntryPoint 合约进行升级需要所有的智能合约账户主体都同意。由于 Entry Point 合约的单点性，ERC4337 在升级上受到了很大限制

- **Dapp 的更新成本**

而如果要使这些 Dapp 完全兼容 ERC4337 钱包，需要改变 Dapp 与钱包交互的审查机制以及代币质押机制

- **跨链**

在以太坊与 Layer2 进行交互的跨链桥中，往往默认接收网络的收款地址为发送网络上付款的地址，因为同一把私钥通过签名便可以实现对两个网络上地址同样的控制效力

- **迁移问题**

账户抽象钱包产品功能参考

- **密钥管理系统：**
 - 「无助记词的」，这个「无」并不是实际意义上的没有密钥，而是指不需要用户备份助记词 / 私钥或者感知到它们的存在。
 - 多重签名逻辑（安全）：只能实现 2/3 和 3/5 等多重签名逻辑；
 - 简单权限管理（顺序）：可以为密钥设置权重，然后为操作账户设置阈值。
 - 基于角色的权限管理（Unipass）：可以为密钥设置权重和角色。不同的角色可以执行不同的操作。每个角色也有相应的阈值。如果超过此阈值，可以执行相应角色的权限。
 - 会话密钥
 - 通行证

- 社交恢复方法
 - 利用 email 的密码学证明 (DKIM Signature) 或者电子护照等生活中常见的密码学工具来增强社交恢复方案的实用性。
- 委托交易
 - 该账户包含用户根据自己偏好定制的相关规则，然后与规则中提到的应用程序进行交互，而无需手动批准每笔交易。
 - 在完全控制钱包的同时，用户可以批准公用事业和订阅账单的自动化支付。
 - 用户可以不断地与dApp交互，而无需每次都进行批准
- gas 费代付：
 - 免 gas fee交易，用户不需要被迫去了解 gas 概念，更不用提前购买各种原生 Token 来支付 gas
 - 自建中继器
 - 设置 Bundler + Paymaster使用指定token支付
- 多链账户同步
- 多链地址统一

账户抽象基础设施

Bundler

- **Stackup** — Golang 实现，目前最完善，各链都有 bundler 业务
- Alchemy — Rust 实现的 Bundler API 客户端，一个 Gas Manager API (可执行 ERC-4337 Paymasters 的功能)，以及一个支持 ERC-4337 标准兼容的所有 RPC 提供方的 Account Abstraction SDK
- biconomy
- Pimlico
- **Infinitism** — TypeScript 实现，由以太坊基金会开发团队出品
- Etherspot 的 Skandha — TypeScript 实现 (UserOperations mempool 的 p2p 网络)

绝大多数 Bundler 的开源性质使其具有非排他性和非竞争性。任何 RPC 端点都可以通过复制开源代码来运行 Bundler，因此难以变现

如果一个 UserOperation 同时被不同的 Bundlers 提交，会出现nonce 碰撞导致交易失败，所有其他 Bundlers 都会因链上失败而损失 Gas

Bundler 可以通过以下方式获得经济利益：

- 执行 UserOperation 后，将最高优先费用与实际 gas 费之间的差额收入囊中。
- 与普通交易的中继器类似，Bundler 可通过对捆绑事务中的 UserOperation 进行排序来获取 MEV。
- 偏向 B 端需求的隐私池，或利用 zkp 实现隐私保护
- 结合MPC或AI技术，达成多签和 AI 风险监测
- 同时基于传统项目的创新也是亮点，如 uniswap 团队的 Universal Paymaster 可以匹配钱包用户和 Paymaster 运营商双方的流动性市场。

Paymaster

Paymaster 一般由项目服务方自行定制部署智能合约，通过与法币存款、兑换、桥接、自动支付、会话、赞助费用和其他功能的结合来改善 dApp 的可用性，有盈利模式

相对于 Bundler 服务来说，Paymaster 服务比较中心化，合约是开源的，但后端是封闭的

功能点：

法币存款：可提供需要链下交易（如法币出入金）的 gas 抽象。如用户可以选择使用信用卡订阅 Paymaster 服务来支付 gas 费。

- Biconomy 和 0xPass 与 Transak 合作提供法币通道；
- Argent Vault 与 Moonpay、Transak 和 Wyre 合作提供法币通道，并具有内置的 DeFi 协议聚合器；
- Etherspot、UniPass 和 Braavos 支持法币通道；

掉期：为防止 gas 波动，可将 Paymaster 集成掉期功能，在特定时间以约定 gas 费用支付；

桥接：如 MetaMask 已经通过和第三方供应商合作的方式把跨链桥集合在钱包中，这些跨链桥可以进一步与 gas abstraction 中的代付合约（Paymaster）进行集合。

- Biconomy 提供跨链桥和跨链通信服务；
- Etherspot、UniPass 和 Braavos 支持 swap 和跨链桥。

会话：可在 Paymaster 集成 Session keys，即用户根据一组参数预先批准一个应用的交易，如一个给定的持续时间，一个最大的 Gas，一个特定代币的最大交易量，或一个特定合约的特定功能等。用例如下

- 在全链游戏中达成用户友好，即不间断地进行游戏，无需每个操作都需签名确认；
- 能够在确认之前设置多个 DeFi 头寸；
- 链上填写多个表格，无需每次都进行确认；
- 在钱包 / 库存中重新安排资产，而不必在每次更改时确认。

多形式代付：通过多种形式的集成甚至可以让 Gas 彻底在用户端「隐身」。

- 开发者代付：应用开发者可以很容易地为他们的用户补贴费用，例如作为一种获客手段；
- UniPass 使用自己的 Relayer 节点支付 gas，计划在未来添加「观看广告免 gas 交易」模式并支持使用跨链桥（bridge）支付 gas。
- 赞助费 / 广告费：可与一些广告商集成，让用户做任务即可免 gas，如点赞某条视频，转发某条推文等；
- 中心化机构：比如与 OKX 交易所结合，绑定 OKX web3 账号与交易所账号，paymaster 可通过扣除交易所账户余额来帮助链上地址进行 gas 代付；
- 多币种 / 方式代付：paymaster 提供与链下流程相关联的 gas 抽象。用户可以使用 ERC-20 代币或信用卡或其他订阅服务等链下支付方式支付 gas 费；
- Biconomy 不仅自行实现了与 EIP 4337 兼容的 Paymaster，还有一个支持任意 ERC20 代币支付 gas 的 Relayer 网络。
- 自动支付：Visa 在 StarkNet 上实施可委托账户解决方案，从而为自托管钱包启用自动支付功能；
- 自定义化的代付逻辑：如 Stackup 的用户也可以定制 gas 代付的逻辑，Stackup 将通过「按需付费」（pay as you go）模式向用户收费。

与 Entry point 的结合：Paymaster 需要在 Entry point 合约上存入以太坊用于支付 UserOperation 的 gas，还需要在 Entry point 合约上额外质押以太坊防止机器人恶意批量创建 Paymaster。这里就存在由于它的质押行为产生的一系列和 Defi 协议的集成机会，如贷款和流动性池等。

AA 与 intent:

intent意图交易，本质上是用户体验层创新、更好更快的理解并分解用户需求，将其变成一个或者多个 UserOperation，telegram bot 是典型的 Intent 创新

Intent 是对特定目标求解其最优 op 路线的过程。过往的 Intent 偏向简单需求，后续可能形成：多条件、多步骤、多执行环境的复杂问题，甚至引入 AI agent。

Intent 机会：操作端可减少签名次数，提升用户体验。应用端可依据意图叙事形成新基建、新语言和新的求解形式，是未来最具潜力的用例之一

- 中间商意图池 or 意图公链：基于意图的应用程序不仅涉及与智能合约交互的新消息格式，还涉及替代内存池形式的传播和交易对手发现机制。设计一种既兼容激励又不集中的意图发现和匹配机制较困难。
- Solver 多样化实现路径：短期已经较成熟的各类 Super Smart Contract 会率先集成，长期看来自由度更高的 AI 将是最理想的形态，但实现难度非常大；Solver 会带来先链下预处理后上链的新范式，利好一些自动化协议（如 Uniswap X）；目前 ZK 协处理器 Axiom 可作为隐私 solve 的一种 demo。
- 新语言来表达意图：目前已有 Juvix 和 Essential，此类项目先发很重要，需要提前构建用户对其的信任；

账户抽象和智能合约钱包和safe的区别：

ERC 4337 的组件并非基于新技术，自 2018 年以来，Argent 等个人钱包服务已经实现了上述交易执行工作流程的一个版本，以支持用户交易的复杂交易执行逻辑。

ERC 4337 的创新之处在于其 Gas 核算和支付的标准化，以及创建一个独立的、抗 DoS 且无需许可的内存池。

账户抽象是一个标准，包含了bundler,payment,endpoint, smw等链上链下组件的标准，智能合约钱包强调链上，safe是一个按自己设计的产品，原生safe偏向去智能合约多签钱包单一解决方案

账户抽象相关EIP

- EIP 5189：通过引入 endoser 合约降低了 Bundler 筛选过程的风险以及进一步防止了 MEV 机器人的恶意攻击
- 4337 衍生产品的辅助：
 - ERC-6551引入了代币绑定账户(TBA)的以太坊标准，赋予NFT能力，使其作为“智能合约账户”发挥作用，并通过无许可注册成为自己的钱包。这实现了一个集成机制，允许所有者在TBA中托管NFT。
- 优化 AA：
 - EIP 6662：辅助完善 AA 账户的实际用例 —— 支持更加用户友好的认证模型，用户的钱包可以成为其登录账户来进行资产管理的 dApp；
 - ERC 6900：模块化账户抽象，提高智能合约的灵活性和便于开发人员的开发；
 - ERC-7579：旨在实现模块开发者和应用程序所需的最小互操作性，同时允许账户供应商进行创新，更为简化的模块化智能账户标准

- ERC 1271 与 ERC 6492：主要解决 4337 领域的验签问题。ERC 6492 相对 1271 做了部分改进，让用户可以在不部署合约的情况下验签；
- EIP 7204 与 EIP 7197：使 AA 钱包有互相定义代币标准的能力。
- 账户迁移辅助：
 - ERC 7377：有助于未来将 EOA 钱包迁移到智能合约钱包中；

钱包案例：

- OKX的账户抽象钱包
- CIRCLE的可编程钱包
- VISA
- Safe的4337模块
- metamask snap

模块化账户抽象

账户抽象（AA）能够使用智能合约代码定制账户，但现有智能合约账户存在问题

- **碎片化**：现在各种功能都以不同的方式实现，无论是通过特定的 SCA 还是独立的插件系统。每个都遵循自己的标准，迫使用户和开发者决定要支持哪些组件。这可能导致平台锁定或重复努力。
- **安全性**：虽然账户和功能之间的灵活性带来了优势，但也可能加剧安全问题。功能可能会被集体审计，但缺乏独立评估可能会增加账户漏洞的风险。
- **可升级性**：随着功能的发展，保留添加、替换或删除功能的能力非常重要。每次更新都重新部署现有功能会引入复杂性。

模块化账户抽象（Modular AA）(MSCA)通过将智能账户与其自定义的功能分离，创建了一个模块化的结构，以开发具有多种功能的安全、无缝集成的钱包

站在用户的角度，模块化方法则是提供了一种更加便捷的方式来扩展针对特定用例的具体功能。模块化帐户允许用户对原有的智能帐户功能进行即时切换、添加或删除，以无需许可的方式组合智能账户模块，以创建专用的 Web3 钱包。用户可对智能账户安全地进行扩展

模块可以启用的各种功能包括：

- 不同签名方案的使用
- 令牌传输时触发特定操作
- 每日支出限额
- 允许支出而无需其他所有者同意的预算
- 通过会话密钥重复交易
- 在特定日期安排和/或自动交易
- 社会恢复
- 黑名单和白名单设

- 角色和权限：为特定账户设置颗粒度级别的特定操作

实现原理

基本原理是将代理合约和逻辑合约分离，通过外部调用（call）或者委托调用（delegatecall），实现智能合约账户的功能模块可组合和可升级

前主要存在两种主流的实现方法：

- SAFE 模块
- ERC-6900

可选的代理模式

最小代理

[EIP-1167](#)标准化了最小代理，是一个轻量级的代理合约模式

这个标准的主要思想是尽可能便宜地部署基础合约的副本，代理合约存储逻辑合约地址，代码只包含一小段用于调用逻辑合约的汇编代码，通过委托调用（delegatecall）它具有与实现合约完全相同的逻辑，且具有自己独立的存储状态，除了部署成本低，还要好处是安全性，将风险隔离。

常见的用例是:Multi-Sig-Wallet、托管账户、某种类型的流动性池等。适用于需要部署多个实例化合约但不希望每个合约都包含重复逻辑的场景

通常，最小代理合约本身并不包含升级逻辑，除非手动设计升级机制。

注意事项

- 有工厂合约来部署最小代理合约
- 通过逻辑合约中的initialize()函数对最小代理的存储进行初始化，将初始化逻辑放到逻辑合约中的initialize()函数中而不是放在最小代理的构造函数中
- initialize()确保只能被调用一次，保障重要变量后续不会被篡改，通常使用openZeppelin的Initializable。
- 需要尽快触发实现合约中的initialize()函数，建议创建最小代理和调用initialize()函数在一笔交易中完成。
- 使实现合约不可用，该合约的唯一目的是充当逻辑合约，而不拥有自己可修改的存储状态。

可升级代理

是一种可以随时更新逻辑合约地址的代理模式，常见的实现包括 OpenZeppelin 的透明可升级代理（Transparent Upgradeable Proxy）和 UUPS（Universal Upgradeable Proxy Standard）模式。

相对于最小代理，可以随时通过特定的管理函数更改指向的逻辑合约地址，这使得逻辑合约可以进行迭代和升级，而无需更改代理合约的地址或部署新的合约实例。在部署成本上会略高一些，常用于需要长期维护和迭代开发的智能合约系统

钻石代理

ERC2535 标准化了 Diamond 代理，这是一种模块化的智能合约系统，可以在部署后可以进行升级/扩展，并且几乎没有大小限制。主代理合约（有状态）Diamond 是一个代理合约，通过委托调用（delegatecall）所谓的切面合约（无状态），自定义逻辑和功能（无状态）模块的切面合约可以隔离存储也可以共享主合约的状态变量。

Safe Modules

safe使用了最小代理，并且增加了可升级的逻辑修改逻辑合约地址

相对于钻石代理的区别：

1. **灵活性**：在启用新插件的情况下，Safe 需要重新部署其单例合约以在其代理中实现更改。相比之下，Diamond 通过其代理中的 diamondCut 函数直接实现这一点，Diamond 引入了更强大的灵活性和模块化。
2. **安全性**：目前，两种结构中都使用了 delegatecall，它可以允许外部代码操纵主合约的存储。在 Safe 架构中，delegatecall 指向一个单一的逻辑合约，而 Diamond 则在多个模块合约（插件）之间使用 delegatecall。因此，恶意插件有可能覆盖另一个插件，从而引入存储冲突的风险，可能损害代理的完整性。

并且增加了审计成本，需要在每次添加新插件时进行全面审计。关键是确保这些插件不会干扰彼此的功能。

因此当safe每当引入新的插件时，需要部署一个新的单例。用户通过升级 Safe 代理合约到所需单例合约，以符合其偏好和要求。

Modules为自定义功能模块，与safe的核心逻辑合约分离，基本的Safe 合约不需要任何模块，调用添加和删除模块操作，需要配置的所有者阈值数量的确认

链下用户先调用Modles，执行自定义逻辑，再通过execTransactionFromModule方法调用到safe代理合约，执行Guards逻辑后执行交易



ERC-6900

采用了钻石代理的思路，提出了模块化智能合约账户和账户插件的概念

当涉及到 AA 的交易过程时，有三个主要过程：验证、执行和 Hook，通过使用代理账户调用模块，可以管理这些过程

将智能合约钱包的功能重新分类为具有三个不同类别（下面列出和定义）的模块化 SCW：

- **验证功能**：可以验证调用者的真实性和帐户的权限
- **执行功能**：允许执行帐户允许的自定义逻辑
- **钩子hooks**：在一个函数执行之前或之后运行的任意逻辑

Function	Description	Name in Safe	Name in ERC6900
Validation	Validate the caller's authenticity and authority to the account	Signature Validator	Validation function
Execution	Execute any custom logic allowed by the account.	Plugin	Execution function
Hook	A module that is executed before or after another function, with the ability to modify state or cause the entire call to revert.	Hook/ Guards	Hook

该标准定义了如何为 MSCA 实现验证、执行和 Hook 插件；合规帐户实施添加、删除、更新和检查插件的方式；辅助类型，例如 FunctionReference

验证功能:

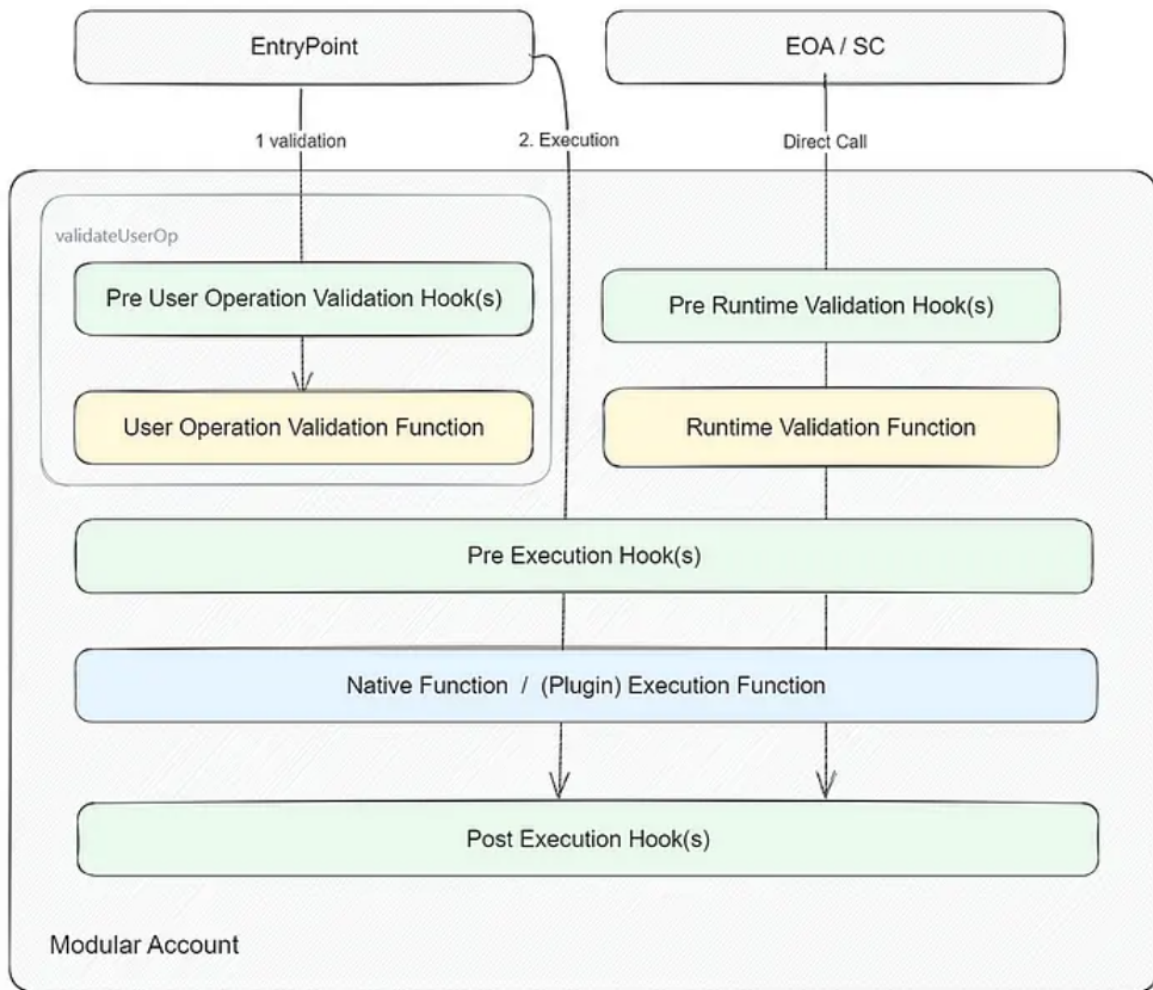
合约账户与用户之间的交互可以分为两种主要类型：

- 第一种是通过 ERC-4337 的入口合约通过用户操作进行交互
- 第二种是EOA用户直接调用合约账户内的函数。

ERC-6900 将这两种类型的交互区分为‘**用户操作验证**’和‘**运行时验证**’，并为每种类型的验证和执行提供单独的流程。

这些验证函数不存在于账户本身；它们都在插件中找到。因此，所有对账户的调用都通过插件中的验证函数。

Modular Account Call Flow



执行功能:

1. 标准执行函数

指与 ERC-4337 参考实现的 `IAccount` 接口兼容的 `execute` 和 `executeBatch` 函数。这些函数可以根据账户的 gas 费用执行所有类型的交互，因此需要严格的验证函数（例如，仅允许所有者调用）。

1. 执行函数

这些是存在于每个插件中的函数和可以从账户执行的常见函数。

Hook:

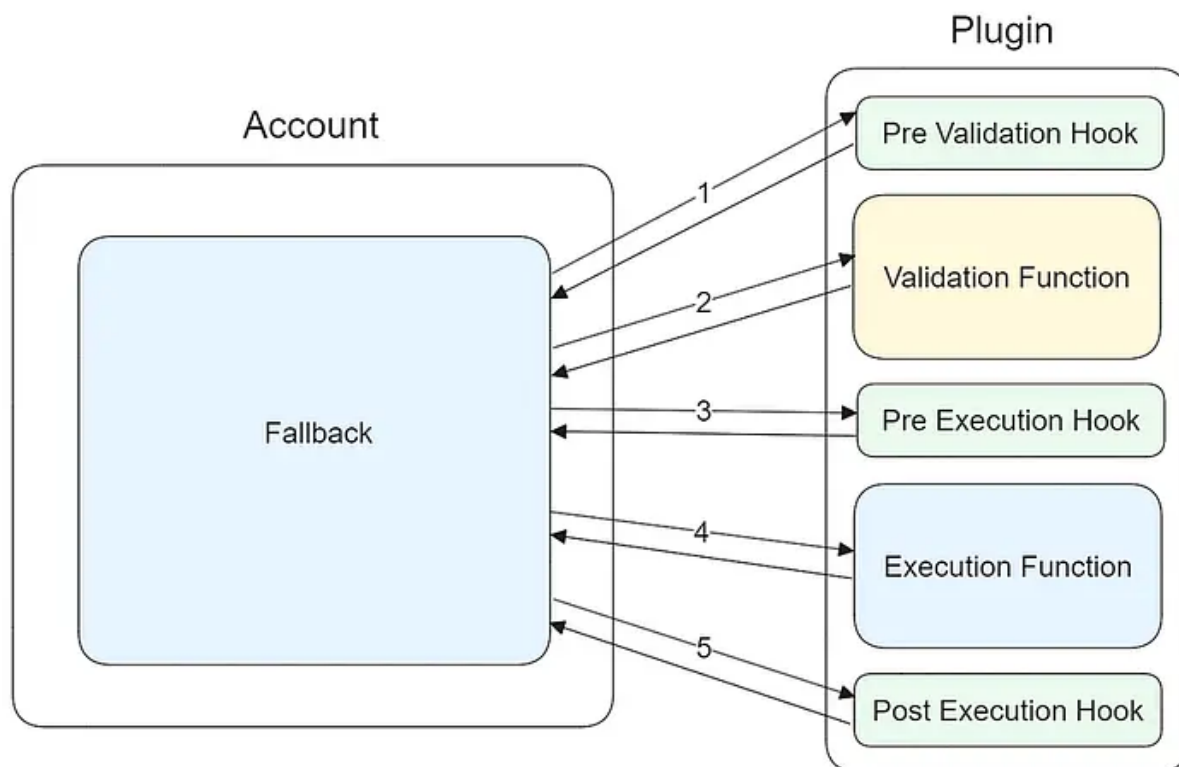
它在其他函数之前和之后运行，定义了需要在特定任务之前和之后执行的操作

- Pre Validation Hook: 验证前的hook

比如如果会话密钥插件中的会话密钥是多重签名钱包，则需要通过多重签名和会话密钥验证。在这种情况下，多重签名的验证可以作为预验证Hook执行，会话密钥的验证作为验证函数执行。

- Pre Execution Hook: 执行前的hook
- Post Execution Hook: 执行后的hook

比如有一个名为 `DailyGasSpendingLimit` 的 Hook，限制每日的 gas 消耗。此Hook可以使用 `gasLeft()` 函数在执行函数之前和之后检查 gas 消耗，并在一天内消耗了一定量的 gas 时阻止函数的执行。



流程：

1. 将要执行的插件的执行函数封装在 `calldata` 中并调用。如果账户中没有与此函数匹配的选择器，则执行 `fallback` 函数（这适用于除 `execute`、`executeBatch` 等标准执行函数之外的所有情况，或稍后提到的 `executeFromPlugin`、`executionFromPluginExternal`）。
2. 在解析哪个插件地址包含 `calldata` 中的执行函数后，执行相关的预验证Hook和验证函数。
如果调用通过 ERC-4337 入口点进行，则验证逻辑已经执行，因此会跳过。
3. 执行与执行函数相关的预执行Hook。此Hook执行的结果返回有关要运行的后执行Hook的信息。
4. 执行执行函数。
5. 根据步骤 3 返回的结果，执行后执行Hook。

权限问题：

在 ERC-6900 中，插件内的函数需要使用 `call` 调用而非Diamond 代理使用 `delegatecall`，并且插件有自己的存储，防止恶意插件可能会删除或操纵账户的存储信息

因此调用中 `msg.sender` 始终是智能合约账户，如果特定插件需要调用另一个插件中的函数，需要严格设置和验证插件可以访问的外部合约函数以及插件之间的交互的权限

模块注册表（模块商店）

提供了模块发现和安全性

现有的智能合约和智能账户的模块化实现依赖于用户和模块开发人员之间的信任假设，模块注册表可以将用户和开发人员之间的信任假设合并到单个实体中

注册表使开发人员能够注册他们的模块，审计员能够验证其安全性，钱包能够集成并让用户能够轻松找到模块并验证其证明信息

最终目标是无需许可的开放注册表。最终，这允许具有不同安全假设的不同各方参与，并且用户可以根据自己的偏好选择信任谁。

库: <https://github.com/erc6900/reference-implementation/tree/develop>

Layer2的账户抽象

L2 和 Gas 成本更低的链上更有可能加速账户抽象 (AA) 的采用

兼容 4337 方案，通过 4337 相关产品进行支持（如 Biconomy多链中继协议、Stackup、GSN抽象用户支付 Gas 等）

Optimism

Arbitrum

Polygon

原生账户抽象方案，即所有的帐户都是智能帐，Starknet 无 Bundler 和 Paymaster，由 Sequencer 确定交易顺序、支付 gas 并执行，zkSync 通过 Operator 确定交易顺序、支付 gas，再调用 bootloader 一起操作；

Starknet: 

Starknet 中仅有合约账户，默认其为顶级账户，不需要再通过复杂的合约层操作便直接可以让合约账户主动发起交易，所有账户必须实现验证和执行功能以验证签名并确保 gas 费。Starknet 禁止验证功能调用外部合约状态，以防止出现未执行的交易。

然而，Starknet 与以太坊还是有些不同，比如缺少 UserOperations、类似 Paymaster 的交易费抽象协议，以及需要具有代币余额的账户来创建新合约。另外，如果已验证的交易失败，Starknet 的排序器就无法收取 gas 费，而以太坊却可以。

在 Starknet 中采用了不同于以太坊的 ECDSA 签名，大大降低了签名和验签过程的成本，从而大大降低了 gas 消耗。。

zkSync: 

zkSync 不区分 EOA 和合约账户。它的账户模型类似于 EIP 4337，并包括独立的 validateTransaction 和 executeTransaction 函数。ZkSync 还支持 Paymasters，可以代付 Gas 费用。zkSync 2.0 中的账户既可以像 EOA 一样发起交易，也可以像智能合约一样实现任意逻辑。

然而，二者还是有不同之处，比如在验证过程中调用已部署的外部合约和外部存储的能力。Paymaster 在交易验证期间也可以调用外部存储。

比较	ERC 4337	Starknet	zkSync
AA 账号	智能合约	协议原生	协议原生
流程逻辑	验证阶段→手续费机制（由 Account Contract 或 Paymaster 支付）→执行阶段		
执行/调用的过程	Bundler → EntryPoint	Sequencer	Operator → bootloader
确定交易顺序的角色	Bundler	Sequencer	Operator
确定 gas 的角色	Bundler	Sequencer	Operator
gas 费消耗	layer1	layer1上链 + layer2	layer1上链 + layer2
是否可以在 Account Contract 部署前送出交易	可以	不可以	不可以
Paymaster 验证规则	通过validatePaymasterOp 和 postOp 定义逻辑，Paymaster 需存入和质押以太坊	无Paymaster	通过validatePaymasterOp 和 postOp 定义逻辑，其中 postOp调用逻辑与以太坊略有不同
是否能调用外部合约	不可以	不可以	可以
如何缓解 DoS 威胁	UserOperation 为 validateUserOp 步骤做了 gas 限制，且 Paymaster 需质押代币	将交易添加到 mempool 并广播之前必须进行本地模拟	只允许触碰自己的 slots，不可以使用到 Global 变量

跨链账户抽象-TODO

账户抽象迁移-TODO

Gnosis Safe

Gnosis Safe多签钱包是以太坊最流行的多签钱包，Safe 广泛应用于个人资产管理、DAO 治理、企业财务等领域，管理近千亿美元资产，合约经过审计和实战测试，支持多链（以太坊，BSC，Polygon等），已融资1亿美金

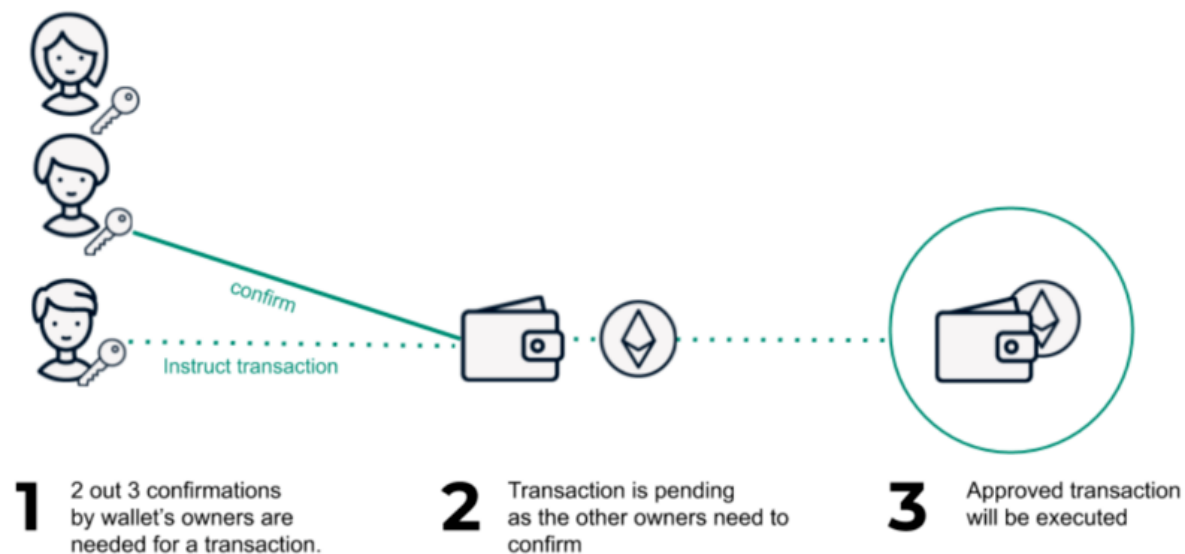
例如：设置为 2-of-3 的 Safe 钱包由三个账户共同拥有，需要至少两个账户的签名才能执行交易。

假设 Alice、Bob 和 Charlie 是该钱包的共有者。Alice 提出向另一地址转账 1 ETH 的提案，并已完成自己的签名，交易提案添加自己的签名。Bob 或 Charlie 在审查提案无误后，可以在交易提案中添加他的签名。一旦签名达到所需数量，任何人均可执行该交易提案，发送到Safe智能合约钱包，但执行该交易的人需要支付 gas 费用。

Safe 不仅支持原生代币（如ETH），还支持各种 ERC20 代币和 ERC721 NFT。用户可以在 Safe 钱包界面直接查看和管理这些资产。

Safe 提供了与多个 DeFi 协议的原生集成，例如：

- Uniswap：直接在 Safe 界面进行代币交换。
- Aave：管理借贷头寸。



使用场景：

企业资金管理

对于企业来说，Safe 提供了一个理想的资金管理解决方案：

- 多人授权：防止单点故障和内部欺诈。
- 交易透明：所有交易都可以在链上追踪和审计。
- 灵活控制：可以为不同级别的支出设置不同的审批流程。

DAO 治理

在去中心化自治组织（DAO）中，Safe 可以作为国库管理工具：

- 提案执行：通过多签机制执行已通过的提案。
- 资金分配：根据社区决策管理和分配资金。
- 权限管理：可以为不同的工作组设置不同的权限。

个人资产保护

对于持有大量加密资产的个人，Safe 提供了额外的安全层：

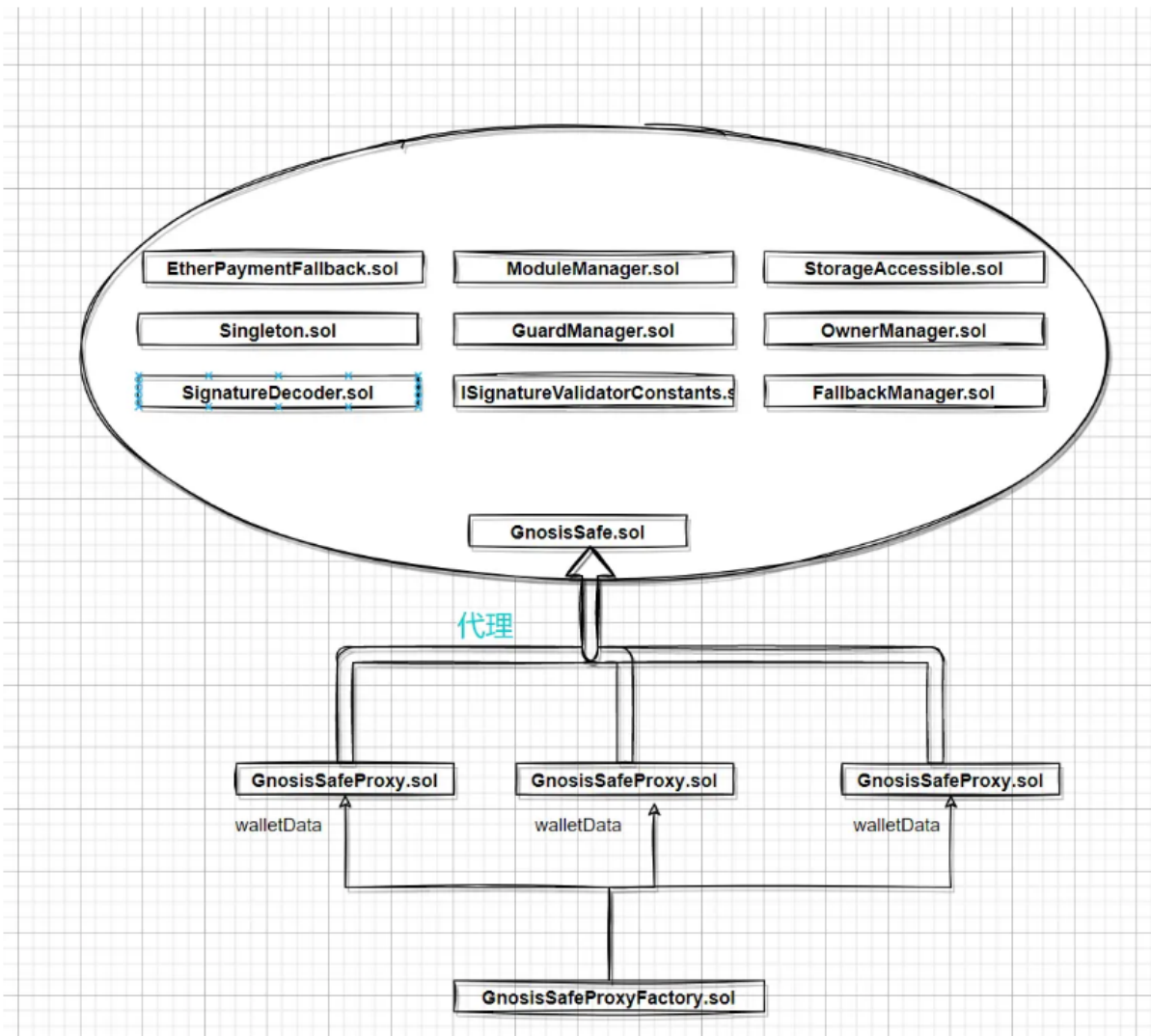
- 防止私钥丢失：即使丢失一个私钥，也不会影响资金安全。
- 继承计划：可以设置时间锁和继承人，确保资产可以被合法继承。
- 分散风险：可以将资产分散到多个设备或地理位置。

Safe合约架构

Gnosis Safe采用模块化架构, 各模块可插拔的, 因此可以根据具有业务需要使用相应模块

- 1. 工厂合约 (GnosisSafeProxy)**：用于创建一个Safe代理合约
- 2. 代理合约 (GnosisSafeProxy)**：每个 Safe 钱包实例都是一个代理合约，指向主合约的实现通过委托调用 (delegatecall)，SafeProxy合约存储了所有者、阈值和实现地址。
- 3. 主合约 (GnosisSafe)**：负责管理safe钱包的核心逻辑，相当于逻辑合约，无状态，包括所有者管理、交易执行等，并定义了集成插件、Hook、函数处理器和签名验证器。

4. **模块管理 (ModuleManager)**：可插拔的功能扩展，如支付流、恢复机制、会话密钥、hook、社交恢复等。
5. **防护模块 (GuardManager)**：用于增加额外的安全检查（制定特定地址才能执行、重入保护、允许/禁止 delegatecall 特定地址等）。



多签流程:

1. 第一个签名者提出交易
 1. 创建交易：定义金额、目的地和任何其他数据
 2. 在提议之前对交易进行链下签名
 3. 将交易和签名提交给safe交易服务
2. 第二位签名者确认交易
 1. 从 Safe 服务获取待处理交易
 2. 执行交易的链下签名
 3. 将签名提交给服务
3. 任何人执行交易
 1. 在此示例中，第一个签名者执行交易
 2. 任何人都可以从 Safe 服务获取待处理的交易
 3. 执行交易的账户支付 gas 费

其中交易服务器可以使用官方服务器也可以自己部署

该服务使多位签名者协同完成多签，通过服务提供的RESTful API接口将签名发送到服务以收集它们，获取有关的交易信息（如读取交易历史记录、待处理交易、启用的模块和保护等）以及其他功能。

具体签名类型

Safe 支持不同类型的签名, 各签名者在链下进行各自的签名,

- ECDSA 签名

该签名让用户可以直接使用小狐狸钱包进行签名

对于EOA账户, 其在链下使用标准的ECDSA椭圆曲线加密算法对EIP-712定义的消息进行签名以批准交易, 签名结果为65字节的

```
{32-bytes r}{32-bytes s}{1-byte v}
```

在链上使用自带的recover函数即可进行签名恢复验证

- EIP-1271签名

该签名让用户可以使用自定义实现的签名算法

IERC1271是一个提供自定义验签名过程的合约对外的接口标准, 传参为待验证消息的hash和签名, 签名的验证过程可以自定义实现, 如果验签通过返回0x1626ba7e, 即 bytes4(keccak256("isValidSignature(bytes32,bytes)"))

```
interface IERC1271 {  
    function isValidSignature(bytes32 hash, bytes memory signature) external  
    view returns (bytes4 magicValue);  
}
```

signature字段在GnosisSafe的编码规则中为

```
{32-bytes signature verifier}{32-bytes data position}{1-byte signature type}  
{32-bytes signature length}{bytes signature data}
```

verifier: 实现 EIP-1271 接口的合约的地址

position: 相对于签名数据开头的偏移量,用于指定签名数据

type: 0, 用于标记为EIP-1271签名类型

length: 指定签名长度

data: 签名数据

safe合约通过外部调用实现IERC1271的合约的 isValidSignature 方法对签名进行验证

```
ISignatureValidator(addr).isValidSignature(data,sig) == EIP1271_MAGIC_VALUE
```

- 预签名

预签名是一种链上签名的解决方案, 其解决的是智能合约没有私钥, 但要对一笔交易进行签名的问题, 签名和验签的过程都是在链上进行的

某个合约调用另一个合约进行

相对其他两种签名, 多了一步, 因此叫预签名

在多签钱包合约中维护一个全局变量

```
mapping(address => mapping(bytes32 => uint256)) public approvedHashes;
```

签名过程实际类似授权的过程, 要进行签名的合约调用钱包approveHash函数, 传入签名数据的hash, 该函数会判断调用者是否属于多签账户中的一个, 是的话将账户和签名数据hash的映射存储起来, 完成签名

```
function approveHash(bytes32 hashToApprove) external {
    require(owners[msg.sender] != 0, "GnosisSafe/approveHash msg.sender is not an owner");
    approvedHashes[msg.sender][hashToApprove] = uint256(1);
}
```

验证签名时, 向mapping查询是否有此映射, 来确认某个账户对一笔交易哈希是否已经预先签名

```
if (v == 1) {
    address currentOwner = address(uint160(uint256(r)));
    require(approvedHashes[currentOwner][dataHash] != 0 || msg.sender == currentOwner);
}
```

GnosisSafe的预签名编码规则:

```
{32-bytes hash validator}{32-bytes ignored}{1-byte signature type}
```

validator: 预签名的账户地址

ignored: 占位符填充0, 该字段未使用

type: 1, 用于标记该签名为预签名

最后

多个签名者在使用各自的签名类型对交易签名后, 执行者将所有签名将合并为一个签名

```
const { SafeFactory } = require('@safe-global/protocol-kit');

const safeFactory = await SafeFactory.init({
  provider: '<provider url>', // 替换为实际的 RPC url
  signer: '<sender private key>', // 私钥，用于部署多签钱包合约，该私钥对应的账户不一定是多签的签名者
})

// SafeAccountConfig结构体中包含了多个签名方以及多签阈值
const safeAccountConfig = {
  owners: [
    '<signer address 1>', // 替换签名者地址
    '<signer address 2>', // 替换签名者地址
    '<signer address 3>', // 替换签名者地址
  ],
  threshold: 2,
}

// 部署多签钱包合约
const safeWallet = await safeFactory.deploySafe({ safeAccountConfig });

// 查看部署的钱包
const safeAddress = await safeWallet.getAddress();
```

```
console.log('Safe 钱包已部署');
console.log(`https://app.safe.global/sep:${safeAddress}`);
```

该过程实际是调用GnosisSafeProxyFactory工厂合约创建了一个新的GnosisSafeProxy钱包代理合约，初始化钱包参数，存储在该代理合约上，并将该代理合约指向一个存在的GnosisSafe实现合约，对GnosisSafeProxy钱包代理合约的所有函数调用都通过 `delegatecall` 的方式远程调用 `GnosisSafe` 逻辑合约。

读取多签钱包相关信息

```
const Safe = require('@safe-global/protocol-kit').default;

const safewallet = await Safe.init({
  provider: '<provider url>', // 替换为实际的 RPC url
  safeAddress: '<safeAddress>', // 替换为实际的多签钱包地址
})

const threshold = await safewallet.getThreshold();
const owners = await safewallet.getOwners();

console.log(`Threshold: ${threshold}`); // 阈值（最小签名数量）
console.log(`Owners: ${owners.join(', ')} `); // 签名者
```

发起一笔 ETH 转账多签交易（前提是该多签钱包有足够的 ETH，提前充值好）

第一位签名者签名

```
const Safe = require('@safe-global/protocol-kit').default;
const SafeApiKit = require('@safe-global/api-kit').default;

const safewallet = await Safe.init({
  provider: '<provider url>', // 替换为实际的 RPC url
  signer: '<signerPrivateKey>', // 替换为多签钱包签名者1的私钥
  safeAddress: '<safeAddress>', // 替换为实际的多签钱包地址
})

const apiKit = new SafeApiKit({
  chainId: await safewallet.getChainId(),
})

// 转账 0.01 ether
const safeTransactionData = {
  to: '<receiver>', // 替换为收款人地址
  data: '0x',
  value: ethers.parseUnits('0.01', 'ether').toString()
}

// 创建多签交易
```

```

const safeTransaction = await safewallet.createTransaction({ 'transactions':
[safeTransactionData] });

// 获取这笔交易的哈希值
const safeTxHash = await safewallet.getTransactionHash(safeTransaction);

// 对交易哈希值进行签名
const senderSignature = await safewallet.signHash(safeTxHash);

// 提交交易到 Safe 服务（这样，其他签名者可以在 Safe 网站中看到待签名的交易）
await apiKit.proposeTransaction({
  'safeAddress': '<safe address>', // 替换为实际的多签钱包地址
  'safeTransactionData': safeTransaction.data,
  'safeTxHash': safeTxHash,
  'senderAddress': new Wallet(signerPrivateKey).address,
  'senderSignature': senderSignature.data,
})

```

第二位签名者签名交易

```

const Safe = require('@safe-global/protocol-kit').default;
const SafeApiKit = require('@safe-global/api-kit').default;

const safewallet = await Safe.init({
  provider: '<provider url>', // 替换为实际的 RPC url
  signer: '<signerPrivateKey>', // 替换为多签钱包签名者2的私钥
  safeAddress: '<safeAddress>', // 替换为实际的多签钱包地址
})

const apiKit = new SafeApiKit({
  chainId: await safewallet.getChainId(),
})

const transaction = (await
apiKit.getPendingTransactions(safeAddress)).results[0];

if (!transaction) return;

const safeTxHash = transaction.safeTxHash;

const signature = await safewallet.signHash(safeTxHash);
const response = await apiKit.confirmTransaction(safeTxHash, signature.data);
console.log('已确认交易:\n', response);

```

执行交易

```

const Safe = require('@safe-global/protocol-kit').default;
const SafeApiKit = require('@safe-global/api-kit').default;

const safewallet = await Safe.init({
  provider: '<provider url>', // 替换为实际的 RPC url
  signer: '<signerPrivateKey>', // 替换为执行交易人的私钥
  safeAddress: '<safeAddress>', // 替换为实际的多签钱包地址
})

```

```
const apiKit = new SafeApiKit({
  chainId: await safewallet.getChainId(),
})

const transaction = (await
apiKit.getPendingTransactions(safeAddress)).results[0];

if (transaction.confirmations.length !== transaction.confirmationsRequired) {
  console.log('未达到签名确认数');
  return;
}

const safeTransaction = await apiKit.getTransaction(transaction.safeTxHash);
const executeTxResponse = await safewallet.executeTransaction(safeTransaction);
const receipt = await executeTxResponse.transactionResponse?.wait();

console.log('交易已执行, hash: ', receipt.hash);
```